

Questions for Initial Lab Engagement — pqc_crypto_module v0.1.0

Disclaimer: Prepared for FIPS 140-3 evaluation, not currently validated.

The following questions are intended for the initial call or written exchange with a prospective FIPS 140-3 accredited laboratory. They are organized by topic to facilitate an efficient discussion.

1. Post-Quantum Algorithm Support

1. **Does your lab currently support CAVP testing for ML-DSA (FIPS 204)?** If not, what is your expected timeline for readiness?
2. **Does your lab currently support CAVP testing for ML-KEM (FIPS 203)?** If not, what is your expected timeline for readiness?
3. **Is the NIST ACVP server currently accepting ML-DSA and ML-KEM algorithm registrations?** If not, do you have visibility into when PQC algorithm support will be available on the ACVP server?
4. **Have you validated any modules that include post-quantum algorithms?** If so, can you share any lessons learned or common findings?

2. CAVP Process

1. **What is the expected timeline for obtaining CAVP algorithm certificates for:**
 - ML-DSA-65 (FIPS 204)
 - ML-KEM-768 (FIPS 203)
 - SHA3-256 (FIPS 202)
2. **What test vector format do you require?** Do you use the NIST ACVP JSON format exclusively, or do you accept other formats?
3. **Can CAVP testing proceed in parallel with FIPS 140-3 module testing,** or must algorithm certificates be obtained first?

3. Rust-Specific Considerations

1. **Do you have prior experience validating Rust-based cryptographic modules?** If so, were there any Rust-specific challenges or considerations?
2. **Are there any concerns with Rust's memory model** (ownership, borrowing, ZeroizeOnDrop) from a FIPS 140-3 perspective?
3. **Do you require specific build environment documentation** beyond what is typical for C/C++ modules? (We can provide `Cargo.toml`, `Cargo.lock`, toolchain version, and deterministic build instructions.)

4. Entropy Source

1. **What is the SP 800-90B compliance path for a module that uses OsRng** (delegating to the operating system's CSPRNG via the `getrandom` system call)?
2. **If the underlying OS CSPRNG has an existing FIPS validation** (e.g., Linux kernel DRBG, macOS CCRNG), is that sufficient for SP 800-90B compliance at the module level, or is additional testing required?
3. **Do you require an entropy assessment report** for the random number generation chain?

5. Timeline and Cost

1. **What is your estimated timeline** from contract signing to submission of the validation report to CMVP, for a FIPS 140-3 Level 1 software module with 3 approved algorithm families?
2. **What is the expected cost range** for the complete engagement (CAVP testing + FIPS 140-3 validation)? Is pricing fixed or time-and-materials?
3. **How many iteration cycles** (findings and re-testing) are typical for a well-documented software module at Security Level 1?
4. **What is your current estimate of the CMVP queue time** from report submission to certificate issuance?

6. Documentation and Process

1. **Do you have a preferred format for the Security Policy and supporting documentation** (Markdown, Word, PDF, etc.)?
2. **Will you review our existing documentation set before formal testing begins?** We have 9 FIPS artifacts ready for review.
3. **What is your onboarding process?** What do you need from us to begin the engagement (NDA, statement of work, initial documentation submission)?

4. **Do you provide a pre-assessment or gap analysis service** before committing to formal validation testing?
-

Module Summary (for lab reference)

- **Module:** `ppc_crypto_module v0.1.0`
- **Type:** Software cryptographic module (Rust crate)
- **Target:** FIPS 140-3 Level 1
- **Approved algorithms:** ML-DSA-65 (FIPS 204), ML-KEM-768 (FIPS 203), SHA3-256 (FIPS 202)
- **Non-approved:** Ed25519, SHA-256, HMAC-SHA256 (runtime-gated, compile-time excludable)
- **Boundary:** 11 source files, single public API module
- **Self-tests:** Power-on KATs for all approved algorithms + continuous RNG test
- **Key zeroization:** `ZeroizeOnDrop` on all private key and shared secret types

Full technical details: see accompanying `module_overview.md` and `executive_summary.md`.